ELDORIA: TREASURES OF THE FALLEN KINGDOM

Memoria del proyecto – Primera evaluación

Autor: Sergio Arévalo Hidalgo
Curso: 1.º de Desarrollo de Aplicaciones Web (DAW)
Lenguaje utilizado: Java

GIT: https://github.com/Checho2525?tab=repositories

Contenido

1. INTRODUCCIÓN 2
2. OBJETIVOS DEL PROYECTO 2
3. ¿EN QUÉ CONSISTE EL JUEGO? 3
   3.1. Estadísticas iniciales del personaje 3
   3.2. Enemigos 3
4. REGLAS Y FUNCIONAMIENTO 4
   4.1. Los combates 4
   4.2. Recompensas y recuperación de vida 4
   4.3. Subir de nivel 4
   4.4. Dificultad de las salas 5
   4.5. Cómo termina una partida 5
5. FUNCIONALIDADES DEL JUEGO 5
6. CÓMO ESTÁ HECHO EL PROGRAMA 6
   6.1. Variables 6
   6.2. Constantes 7
   6.3. Arrays 7
   6.4. Condicionales 7
   6.5. Bucles 8
   6.6. Entrada de datos 8
   6.7. Números aleatorios 8
7. PRUEBAS DEL PROGRAMA 8
8. PROBLEMAS ENCONTRADOS Y SOLUCIONES 9
   8.1. Mantener el menú activo 9
   8.2. Terminar la partida al morir 9

8.3. Hacer funcionar la defensa 9
8.4. Controlar las opciones incorrectas 10
8.5. Sistema de experiencia 10
9. IDEAS PARA FUTURAS VERSIONES 10
10. CONCLUSIÓN 11

---

1. INTRODUCCIÓN

Para este proyecto he desarrollado un pequeño videojuego llamado **Eldoria: Treasures of the Fallen Kingdom**. Es un juego de rol (RPG) hecho en Java y que funciona desde la consola.

La idea del juego es controlar a un personaje que tiene que ir avanzando por diferentes salas y enfrentándose a enemigos. Cuanto más avanzas, más difíciles se vuelven los combates.

Durante las partidas puedes atacar, defenderte o intentar huir. Si consigues derrotar a un enemigo, recibes oro y experiencia. También recuperas algo de vida y, cuando consigues suficiente experiencia, puedes subir de nivel y mejorar tu personaje.

El objetivo es llegar lo más lejos posible. No hay una última sala ni un jefe final en esta primera versión. La dificultad va aumentando hasta que el personaje se queda sin vida y termina la partida.

2. OBJETIVOS DEL PROYECTO

El objetivo principal era crear un juego sencillo en Java para poner en práctica lo que hemos aprendido durante la primera evaluación.

Con este proyecto quería conseguir varias cosas:

- Crear un menú que permita moverse por las diferentes opciones del juego.
- Hacer combates por turnos contra distintos enemigos.
- Permitir que el jugador pueda atacar, defenderse y huir.
- Añadir recompensas para conseguir oro y experiencia.
- Hacer que el personaje pueda subir de nivel y mejorar sus estadísticas.
- Conseguir que los enemigos sean cada vez más fuertes según avanzas.

- Utilizar arrays para guardar los datos del personaje y de los enemigos.
- Controlar las opciones incorrectas y evitar que el juego siga funcionando cuando el personaje pierde toda su vida.

También quería que el juego tuviese una base que se pueda ampliar más adelante con nuevas características.

3. ¿EN QUÉ CONSISTE EL JUEGO?

Al abrir Eldoria aparece un menú principal con tres opciones:

1. Comenzar partida.
2. Consultar estadísticas.
3. Salir.

Si eliges comenzar partida, el juego selecciona aleatoriamente uno de los tres enemigos disponibles: Slime, Goblin o Esqueleto.

Cada enemigo tiene sus propias estadísticas y, dependiendo de la sala en la que estés, tendrá más vida, hará más daño y dará más oro.

Una vez empieza el combate, puedes elegir qué hacer en cada turno. Si derrotas al enemigo, recibes tus recompensas y avanzas a la siguiente sala.

La partida continúa hasta que decides salir o tu personaje se queda sin vida.

3.1. Estadísticas iniciales del personaje

El personaje empieza con estas estadísticas:

Estadística	Valor inicial
Vida	50
Daño	10
Oro	0
Nivel	1
Experiencia	0

La vida indica cuánto puede aguantar el personaje, el daño sirve para calcular cuánto quita al enemigo y el oro representa las recompensas conseguidas.

3.2. Enemigos

En esta primera versión hay tres enemigos diferentes:

Enemigo	Vida	Daño	Oro

Slime	20	7	5
Goblin	20	6	6
Esqueleto	25	7	7

El enemigo que aparece en cada combate se elige al azar, por lo que no siempre te enfrentarás al mismo.

4. REGLAS Y FUNCIONAMIENTO

4.1. Los combates

Los combates funcionan por turnos. Primero eliges una acción y después, si el enemigo sigue vivo, te ataca.

Hay tres acciones disponibles:

- **Atacar:** haces daño al enemigo según el daño que tenga tu personaje.
- **Defenderse:** reduces a la mitad el daño que recibes del siguiente ataque enemigo.
- **Huir:** abandonas el combate y vuelves al menú principal. No recibes recompensas por ese combate.

Si introduces una opción que no existe, el juego te avisa y te vuelve a pedir que elijas. El enemigo tampoco te ataca por haber introducido una opción incorrecta.

4.2. Recompensas y recuperación de vida

Cuando derrotas a un enemigo consigues oro y 10 puntos de experiencia.

También recuperas 10 puntos de vida. Si esa recuperación hace que superes tu vida máxima, el programa ajusta la vida al máximo permitido.

Es importante tener en cuenta que la recuperación de vida por derrotar a un enemigo y la recuperación que se consigue al subir de nivel son dos cosas diferentes.

4.3. Subir de nivel

Para subir de nivel necesitas conseguir experiencia. Al principio hacen falta 20 puntos.

Cuando subes de nivel ocurren varias cosas:

- Aumentas un nivel.
- Se descuentan los puntos de experiencia necesarios.
- Necesitas 10 puntos más de experiencia para el siguiente nivel.

- Tu vida máxima aumenta en 10 puntos.
- Tu daño aumenta en 4 puntos.
- Recuperas otros 10 puntos de vida.

Por ejemplo, al pasar del nivel 1 al nivel 2, la vida máxima pasa de 50 a 60 y el daño aumenta de 10 a 14.

4.4. Dificultad de las salas

Los enemigos se vuelven más fuertes según avanzas por las salas.

Por cada sala que avanzas, sus estadísticas aumentan:

- 5 puntos más de vida.
- 2 puntos más de daño.
- 2 monedas más de oro como recompensa.

Por ejemplo, un Slime de la segunda sala tendrá 25 puntos de vida, hará 9 puntos de daño y dará 7 monedas de oro.

4.5. Cómo termina una partida

El juego termina cuando la vida del personaje llega a cero.

En esta versión no hay una victoria final, porque la idea es intentar llegar cada vez más lejos y superar el máximo número de salas posible.

Si el jugador decide huir, vuelve al menú principal y puede comenzar otro combate. Si pierde toda su vida, la partida termina y el programa finaliza.

5. FUNCIONALIDADES DEL JUEGO

Estas son las principales funciones que tiene esta primera versión:

Menú principal: permite comenzar una partida, consultar las estadísticas o salir.

Enemigos aleatorios: cada combate puede ser contra un Slime, un Goblin o un Esqueleto.

Combate por turnos: puedes elegir entre atacar, defenderte o huir.

Oro y experiencia: al ganar un combate consigues recompensas.

Recuperación de vida: después de derrotar a un enemigo recuperas 10 puntos de vida, sin superar el máximo.

Subida de nivel: puedes mejorar la vida máxima y el daño de tu personaje.

Dificultad progresiva: los enemigos se hacen más fuertes al avanzar por las salas.

Consulta de estadísticas: puedes ver la vida, el daño, el oro, el nivel, la experiencia y la sala actual.

Colores en consola: he añadido colores para que los mensajes sean más fáciles de distinguir y el juego no se vea tan simple.

6. CÓMO ESTÁ HECHO EL PROGRAMA

El programa está desarrollado en Java y, de momento, todo el código se encuentra dentro de una clase llamada Eldoria.

Para esta primera versión he utilizado los elementos básicos de Java que hemos ido viendo en clase, como variables, constantes, arrays, condicionales y bucles.

6.1. Variables

He utilizado diferentes variables para guardar la información necesaria durante la partida.

Algunas de las más importantes son:

Variable	**Para qué sirve**
respuesta	Guarda la opción elegida en el menú.
accion	Guarda lo que quiere hacer el jugador durante el combate.
nivel	Indica el nivel actual del personaje.
experiencia	Guarda la experiencia conseguida.
experienciaNecesaria	Indica cuánta experiencia hace falta para subir de nivel.
vidaMaxima	Guarda el máximo de vida que puede tener el personaje.
sala	Indica por qué sala va el jugador.
vivo	Sirve para controlar si la partida continúa.
enemigoAleatorio	Permite seleccionar al enemigo al azar.
vidaEnemigo	Guarda la vida actual del enemigo.
dañoEnemigo	Guarda el daño que hace el enemigo.
oroEnemigo	Guarda el oro que se consigue al derrotarlo.

defenderse	Indica si el jugador ha elegido defenderse.

6.2. Constantes

También he utilizado constantes con final.

En este caso las he usado para guardar los códigos de los colores de la consola, como el rojo, verde, amarillo, azul o morado.

He utilizado nombres en mayúsculas para diferenciarlas de las variables normales.

6.3. Arrays

Los arrays son una parte importante del programa porque los utilizo para guardar las estadísticas del personaje y de los enemigos.

Por ejemplo, el personaje se guarda de esta manera:

int[] personaje = {50, 10, 0};

Cada posición guarda un dato diferente:

Posición	**Dato**
0	Vida
1	Daño
2	Oro

También tengo tres arrays para guardar los datos del Slime, el Goblin y el Esqueleto.

De esta forma puedo acceder a cada estadística utilizando su posición. Por ejemplo, personaje[0] sirve para consultar la vida y personaje[2] para consultar el oro.

Los arrays tienen una función real dentro del juego, ya que sus valores se utilizan para preparar los combates y actualizar las estadísticas del personaje.

6.4. Condicionales

He utilizado if, else if y else para controlar las diferentes situaciones del juego.

Por ejemplo, sirven para comprobar qué opción ha elegido el jugador, seleccionar al enemigo, calcular los ataques y comprobar si alguien ha sido derrotado.

También se utilizan para controlar la subida de nivel, la defensa y las opciones incorrectas.

6.5. Bucles

Principalmente utilizo dos bucles while.

Uno se encarga de mantener abierto el menú principal hasta que el jugador decide salir o pierde la partida.

El otro mantiene el combate activo mientras el jugador y el enemigo sigan teniendo vida.

También utilizo break para salir del combate cuando el jugador huye o es derrotado, y continue para repetir el turno cuando introduce una opción incorrecta.

6.6. Entrada de datos

Para que el jugador pueda elegir las opciones utilizo Scanner.

Esta herramienta permite leer los números que introduce el usuario por teclado mediante nextInt().

Al terminar el programa cierro el Scanner con scanner.close().

En esta versión las opciones se validan comprobando que los números estén dentro de los valores permitidos. La entrada de letras todavía no está controlada y es una posible mejora futura.

6.7. Números aleatorios

Para que los enemigos no aparezcan siempre en el mismo orden utilizo Math.random().

Con esto genero un número aleatorio entre 0 y 2, que utilizo para decidir qué enemigo aparece en cada combate.

7. PRUEBAS DEL PROGRAMA

Para comprobar que el juego funciona correctamente, he preparado diferentes pruebas que permiten revisar las opciones principales.

Nº	Prueba	Resultado esperado
1	Comenzar partida	Aparece un enemigo y empieza el combate.
2	Consultar estadísticas	Se muestran los datos del personaje.
3	Salir	El programa termina.
4	Introducir una opción incorrecta en el menú	Aparece un mensaje de error.

5	Atacar	El enemigo pierde vida.
6	Defenderse	El jugador recibe la mitad del daño, redondeado hacia abajo.
7	Huir	Se abandona el combate y se vuelve al menú.
8	Introducir una acción incorrecta	Se vuelve a pedir una acción sin recibir daño.
9	Derrotar a un enemigo	Se recibe oro, experiencia y vida.
10	Subir de nivel	Mejoran las estadísticas del personaje.
11	Avanzar de sala	Los enemigos tienen mejores estadísticas.
12	Llegar a cero de vida	Aparece el mensaje de derrota y termina el programa.

8. PROBLEMAS ENCONTRADOS Y SOLUCIONES

Durante el desarrollo me he encontrado con algunos problemas, sobre todo a la hora de controlar los turnos y conseguir que las diferentes partes del juego funcionasen correctamente.

8.1. Mantener el menú activo

Era necesario controlar que el menú no apareciese una sola vez y que el programa pudiese seguir funcionando después de cada acción.

Para solucionarlo utilicé un bucle while, que mantiene el menú activo hasta que se cumple una condición de salida.

8.2. Terminar la partida al morir

Otro punto importante era conseguir que el juego terminase cuando el personaje se quedase sin vida.

Para ello añadí una variable booleana llamada vivo. Cuando la vida llega a cero, esta variable cambia a false y el bucle principal deja de ejecutarse.

8.3. Hacer funcionar la defensa

Para controlar la defensa utilicé una variable booleana llamada defenderse.

Cuando el jugador elige esta opción, el daño recibido se divide entre dos. También tuve que controlar que la defensa no se mantuviese activa durante todos los combates, por lo que se reinicia en cada turno.

8.4. Controlar las opciones incorrectas

También tuve que solucionar que el enemigo atacase cuando el jugador introducía una opción que no existía.

Lo solucioné utilizando continue, que hace que el programa vuelva a solicitar una acción sin ejecutar el ataque del enemigo.

8.5. Sistema de experiencia

Otra parte que necesitaba controlar era la subida de nivel. Había que comprobar cuándo se alcanzaba la experiencia necesaria y aplicar correctamente las mejoras.

Para ello utilizo una condición que comprueba si la experiencia acumulada es suficiente y, si lo es, actualiza el nivel y las estadísticas.

9. IDEAS PARA FUTURAS VERSIONES

Esta primera versión tiene las mecánicas principales, pero todavía hay bastantes cosas que me gustaría añadir más adelante.

Algunas ideas son:

- Crear diferentes clases para el personaje y los enemigos.
- Añadir un inventario para guardar objetos.
- Incorporar armas y armaduras que mejoren las estadísticas.
- Añadir pociones para recuperar vida.
- Crear habilidades y ataques especiales.
- Añadir una tienda para gastar el oro conseguido.
- Incorporar nuevos tipos de enemigos y jefes.
- Permitir elegir diferentes razas o tipos de personaje.
- Separar el código en varias clases para tener el proyecto mejor organizado.
- Controlar también las entradas de texto incorrectas, como cuando el usuario introduce letras en lugar de números.
- Guardar las partidas para poder continuar más adelante.

Estas mejoras se podrían ir añadiendo en las siguientes evaluaciones, utilizando los nuevos conocimientos que vayamos aprendiendo.

10. CONCLUSIÓN

Con este proyecto he podido poner en práctica bastantes de los conceptos de Java que hemos aprendido durante la primera evaluación.

He trabajado con variables, constantes, arrays, condicionales, bucles y entrada de datos. Además, he tenido que pensar cómo organizar las diferentes partes del juego para que todo funcionase correctamente.

Una de las partes que más trabajo me ha llevado ha sido el sistema de combate, porque había que controlar los turnos, los ataques, la defensa, las recompensas y la derrota.

Aunque Eldoria todavía es una versión bastante básica, ya tiene una base jugable y se puede seguir ampliando en el futuro.

En general, estoy satisfecho con el resultado de esta primera versión, porque he conseguido crear un juego desde cero y aplicar los conocimientos de programación en algo práctico.